BÁO CÁO BÀI THỰC HÀNH SỐ 4
MÔN PHÁT TRIỂN ỨNG DỤNG IOT

Giảng viên	: TS.Nguyễn Quốc Uy
Họ và tên sinh viên	: Trương Đức Mạnh
Mã sinh viên	: B22DCPT162
Nhóm lớp	: 02

Hà Nội - 2026

LỜI CẢM ƠN

Em xin bày tỏ lòng tri ân sâu sắc tới các thầy cô khoa Công nghệ Đa phương tiện – Học viện Công nghệ Bưu chính Viễn thông. Những kiến thức và phương pháp luận quý báu mà thầy cô truyền dạy chính là hành trang quan trọng để em tự tin bước tiếp trên con đường sự nghiệp sau này.

Đặc biệt, em xin gửi lời cảm ơn chân thành nhất tới thầy Nguyễn Quốc Uy. Với sự tâm huyết và chỉ bảo tận tình của thầy trong suốt học phần Phát triển ứng dụng IoT, em đã có cơ hội tiếp cận và hiện thực hóa nhiều kiến thức thực tiệm để hoàn thành bài tập lớn này.

Dù đã dành nhiều tâm sức, nhưng do kiến thức còn hạn hẹp nên bài báo cáo khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu từ thầy để đề tài được hoàn thiện hơn.

Em xin trân trọng cảm ơn.

MỤC LỤC

LỜI CẢM ƠN 2
MỤC LỤC 3
MỤC LỤC HÌNH ẢNH 4
CHƯƠNG I: TỔNG QUAN 5
1.1 Đặt vấn đề 5
1.2 Mục tiêu đề tài 5
1.3 Phạm vi hệ thống 5
1.4 Tổng quan các công nghệ sử dụng 5
CHƯƠNG II: THIẾT KẾ HỆ THỐNG 7
2.1. Kiến trúc hệ thống 7
2.2 Usecase 10
CHƯƠNG III: XÂY DỰNG HỆ THỐNG 14
3.1 Thiết kế cơ sở dữ liệu 14
3.2 Thiết kế API 15
3.3 Triển khai phần cứng 16
3.4 Xây dựng Backend 20
3.5 Xây dựng Frontend 22
3.6 API document (Postman) 24
CHƯƠNG IV: KẾT QUẢ THỰC HIỆN 26
4.1 Giao diện Dashboard 26
4.2 Giao diện DataSensor 27
4.3 Giao diện ActionHistory 28
4.4 Giao diện Profile 29
CHƯƠNG IV: KẾT LUẬN 30
5.1 Kết quả đạt được 30
5.2 Hạn chế 30
5.3 Hướng phát triển tương lai 30

MỤC LỤC HÌNH ẢNH

Hình 2.1 Kiến trúc hệ thống.....7
Hình 2.2 Luồng theo dõi cảm biến.....10
Hình 2.3 Luồng điều khiển thiết bị.....11
Hình 24 Luồng tra cứu lịch sử thao tác.....12
Hình 2.5 Luồng tra cứu dữ liệu cảm biến.....12
Hình 3.1 Thiết kế cơ sở dữ liệu.....14
Hình 3.2 Triển khai phần cứng thực tế.....16
Hình 3.3 Cấu hình hết nối phần cứng.....17
Hình 3.4 Tiếp nhận và xử lý các lệnh điều khiển.....18
Hình 3.5 Thu thập và truyền dữ liệu cảm biến.....19
Hình 3.6 Cấu trúc backend.....20
Hình 3.7 Cấu trúc frontend.....22
Hình 3.8 API document (Điều khiển thiết bị).....24
Hình 3.9 API document (Tìm kiếm dữ liệu cảm biến.....24
Hình 3.10 API document (Tìm kiếm lịch sử điều khiển).....25
Hình 4.1 Giao diện Dashboard thực tế.....26
Hình 4.2 Giao diện DataSensor thực tế.....27
Hình 4.3 Giao diện ActionHistory thực tế.....28
Hình 4.4 Giao diện Profile thực tế.....29

CHƯƠNG I: TỔNG QUAN

1.1 Đặt vấn đề

Trong bối cảnh cách mạng công nghiệp 4.0, Internet vạn vật (IoT) đã trở thành một trong những trụ cột quan trọng nhất, làm thay đổi cách thức con người tương tác với môi trường xung quanh. Việc tự động hóa và giám sát môi trường sống không còn là một khái niệm xa lạ mà đã trở thành nhu cầu thiết yếu nhằm nâng cao chất lượng cuộc sống, tối ưu hóa sự tiện nghi và đặc biệt là tiết kiệm nguồn năng lượng đang dần cạn kiệt.

Thực tế cho thấy, việc quản lý các thiết bị điện và theo dõi thông số môi trường theo cách thủ công thường gây ra sự lãng phí và thiếu tính kịp thời. Đề tài tập trung giải quyết bài toán quản lý và điều khiển từ xa các thông số môi trường (nhiệt độ, độ ẩm, ánh sáng) cũng như trạng thái các thiết bị điện (quạt, điều hòa, đèn) trong mô hình phòng thông minh thông qua một hệ thống tích hợp đa nền tảng.

1.2 Mục tiêu đề tài

Dự án hướng tới việc xây dựng một hệ thống hoàn thiện với các mục tiêu cụ thể sau:

Giám sát thời gian thực (Real-time Monitoring): Thu thập dữ liệu từ các cảm biến môi trường và hiển thị chúng lên giao diện người dùng với độ trễ thấp nhất.

Điều khiển linh hoạt (Remote Control): Cho phép người dùng bật/tắt các thiết bị điện như quạt, điều hòa, và đèn thông qua giao diện Web ở bất kỳ đâu có kết nối Internet.

Quản lý dữ liệu bền vững: Xây dựng hệ thống lưu trữ lịch sử đo đạc của cảm biến và nhật ký thao tác điều khiển thiết bị, phục vụ cho việc tra cứu, thống kê và phân tích sau này.

1.3 Phạm vi hệ thống

Hệ thống được thiết kế tập trung vào trải nghiệm người dùng thông qua các trang quản lý chuyên biệt:

Dashboard: Đóng vai trò là trung tâm điều khiển, hiển thị các biểu đồ thông số thay đổi theo thời gian và các nút bấm tương tác trực quan với thiết bị (Fan, Air Condition, Light Bulb).

Data Sensor: Chuyên biệt cho việc lưu trữ và hiển thị chi tiết lịch sử các giá trị từ cảm biến LM35, DHT22 và Quang trở (LDR).

Action History: Nhật ký hệ thống ghi nhận mọi hành động của người dùng và trạng thái thực thi lệnh (SUCCESS/FAILED), giúp người dùng dễ dàng kiểm soát các thao tác đã thực hiện.

1.4 Tổng quan các công nghệ sử dụng

Hệ thống được xây dựng trên sự kết hợp của các công nghệ hiện đại, đảm bảo tính ổn định và khả năng mở rộng:

1.4.1 Backend: Spring Boot

Spring Boot: Là một framework mạnh mẽ dựa trên Java, giúp đơn giản hóa quá trình phát triển ứng dụng web thông qua cấu hình tự động. Nó cung cấp hiệu năng cao và khả năng bảo mật tốt cho các hệ thống doanh nghiệp.

Spring Data JPA: Hỗ trợ tương tác với cơ sở dữ liệu một cách dễ dàng thông qua các phương thức ánh xạ thực thể (ORM), giảm thiểu việc viết mã SQL thủ công.

Spring WebSocket & STOMP: Công nghệ cho phép giao tiếp hai chiều giữa Server và Client theo thời gian thực. Đây là yếu tố then chốt để cập nhật dữ liệu cảm biến lên giao diện ngay lập tức mà không cần người dùng tải lại trang.

Spring Integration MQTT: Cầu nối chuyên dụng để Backend giao tiếp với các thiết bị phần cứng thông qua giao thức MQTT.

1.4.2 Frontend: React & TypeScript

React: Thư viện JavaScript hàng đầu của Meta, chuyên dụng để xây dựng giao diện người dùng theo hướng thành phần (component-based), giúp giao diện phản hồi nhanh và mượt mà.

TypeScript: Một phiên bản mở rộng của JavaScript bổ sung thêm tính năng kiểm tra kiểu dữ liệu tĩnh, giúp giảm thiểu lỗi trong quá trình phát triển và làm cho mã nguồn dễ bảo trì hơn.

Recharts: Thư viện biểu đồ chuyên sâu giúp trực quan hóa dữ liệu cảm biến dưới dạng đồ thị đường, đồ thị cột sinh động.

1.4.3 Cơ sở dữ liệu: MySQL

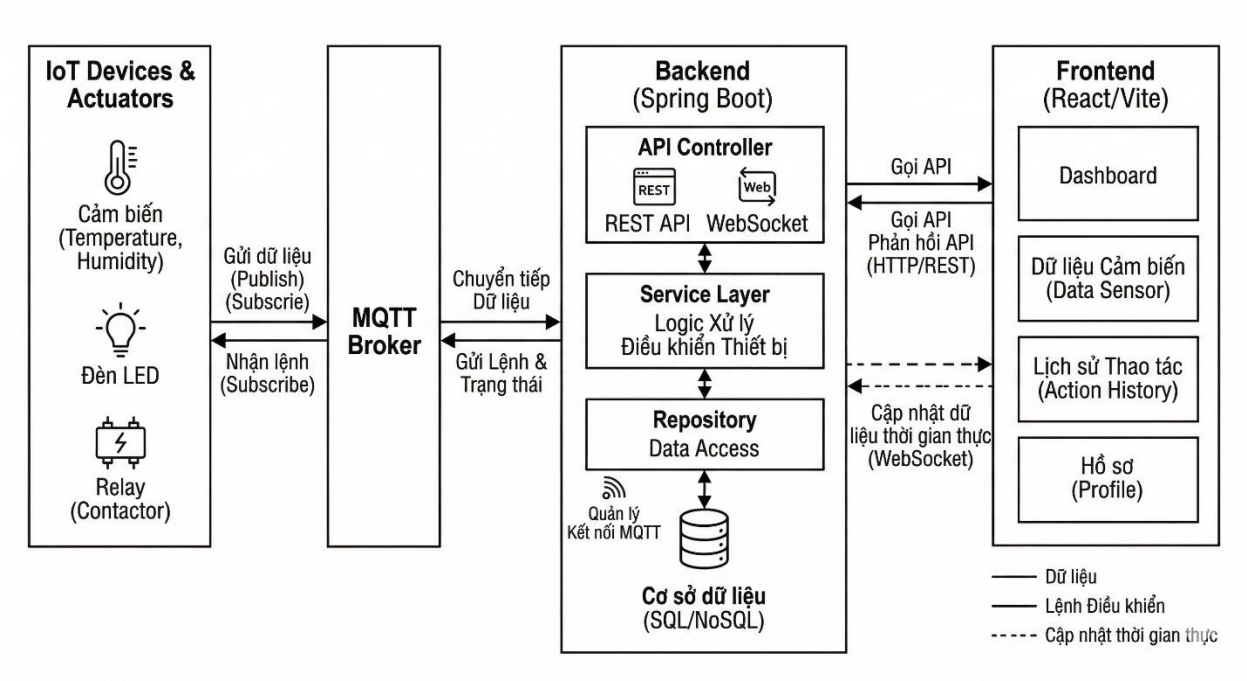
MySQL: Hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) phổ biến nhất thế giới nhờ tính ổn định, tốc độ truy vấn nhanh và khả năng lưu trữ hàng triệu bản ghi dữ liệu cảm biến một cách an toàn.

1.4.4 Giao thức truyền thông: MQTT

MQTT (Message Queuing Telemetry Transport): Là giao thức truyền tin nhẹ nhất hiện nay, hoạt động theo mô hình Xuất bản/Đăng ký (Publish/Subscribe). MQTT đặc biệt phù hợp cho các thiết bị IoT có băng thông hạn chế và năng lượng thấp, đảm bảo việc gửi lệnh và nhận dữ liệu diễn ra gần như tức thời

CHƯƠNG II: THIẾT KẾ HỆ THỐNG

2.1. Kiến trúc hệ thống



Hình 2.1 Kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình phân lớp chức năng, đảm bảo tính module hóa cao, giúp dễ dàng mở rộng và bảo trì. Kiến trúc tổng thể bao gồm 4 thành phần cốt lõi:

2.1.1. Lớp thiết bị

Phần cứng đóng vai trò là lớp thu thập và thực thi trong hệ thống IoT, bao gồm vi điều khiển ESP32 kết nối với các cảm biến và thiết bị chấp hành.

- Chức năng chính:
  - Thu thập dữ liệu từ các cảm biến (nhiệt độ, độ ẩm, ánh sáng).
  - Tiền xử lý dữ liệu (lọc nhiễu, chuẩn hóa).
  - Gửi dữ liệu lên hệ thống Backend thông qua giao thức MQTT.
  - Nhận lệnh điều khiển từ Backend để thực thi (bật/tắt thiết bị).
- Đặc điểm:
  - Hoạt động liên tục, tiêu thụ năng lượng thấp.
  - Có khả năng kết nối WiFi hoặc Ethernet.
  - Hỗ trợ giao thức MQTT client.

2.1.2 Hạ tầng truyền thông

Hệ thống sử dụng giao thức MQTT (Message Queuing Telemetry Transport) làm phương thức giao tiếp chính giữa phần cứng và Backend.

- Cơ chế hoạt động:
  - MQTT hoạt động theo mô hình Publish/Subscribe.
  - Thiết bị IoT (client) sẽ:
    - Publish dữ liệu cảm biến lên các topic sensor/data.
    - Subscribe topic device/control để nhận lệnh điều khiển.
  - Backend đóng vai trò là Subscriber và Publisher trung gian
- Thành phần trung tâm:
  - MQTT Broker (ví dụ: Mosquitto, EMQX)
    - Quản lý việc phân phối message giữa các client.
    - Đảm bảo message được truyền đúng topic.
- Ưu điểm:
  - Nhẹ, tối ưu cho thiết bị tài nguyên thấp.
  - Độ trễ thấp, phù hợp hệ thống realtime.
  - Hỗ trợ QoS (Quality of Service) để đảm bảo độ tin cậy.

2.1.3 Backend

Backend được phát triển bằng Spring Boot, đóng vai trò là trung tâm xử lý (Processing Layer) của toàn hệ thống.

- Chức năng chính:
  - Nhận dữ liệu từ MQTT Broker.
  - Xử lý logic nghiệp vụ (business logic).
  - Lưu trữ dữ liệu vào cơ sở dữ liệu MySQL.
  - Cung cấp API cho Frontend (RESTful API).
  - Gửi dữ liệu realtime đến Frontend qua WebSocket.
- Các module chính:
  - MQTT Client Module:
    - Subscribe dữ liệu từ các topic.
    - Publish lệnh điều khiển xuống thiết bị.

    - Service Layer:
        - Xử lý dữ liệu, validate, rule logic.
    - Repository Layer:
        - Tương tác database (JPA/Hibernate).
    - WebSocket Server:
        - Đẩy dữ liệu realtime lên UI.
- Đặc điểm:
    - Xử lý bất đồng bộ.
    - Dễ mở rộng.
    - Tích hợp tốt với hệ sinh thái Java.

2.1.4 Frontend

Frontend được xây dựng bằng **React kết hợp với TypeScript, cung cấp giao diện trực quan cho người dùng.

- Chức năng chính:
    - Hiển thị dữ liệu cảm biến theo thời gian thực.
    - Điều khiển thiết bị từ xa (bật/tắt, cấu hình).
    - Hiển thị biểu đồ thống kê (charts).
    - Quản lý người dùng và thiết bị.
- Công nghệ sử dụng:
    - React: xây dựng UI component-based.
    - TypeScript: tăng tính an toàn kiểu dữ liệu.
    - WebSocket client: nhận dữ liệu realtime.
    - REST API: giao tiếp với Backend.
- Đặc điểm:
    - Giao diện responsive.
    - Tương tác nhanh, realtime.
    - Dễ mở rộng và bảo trì.

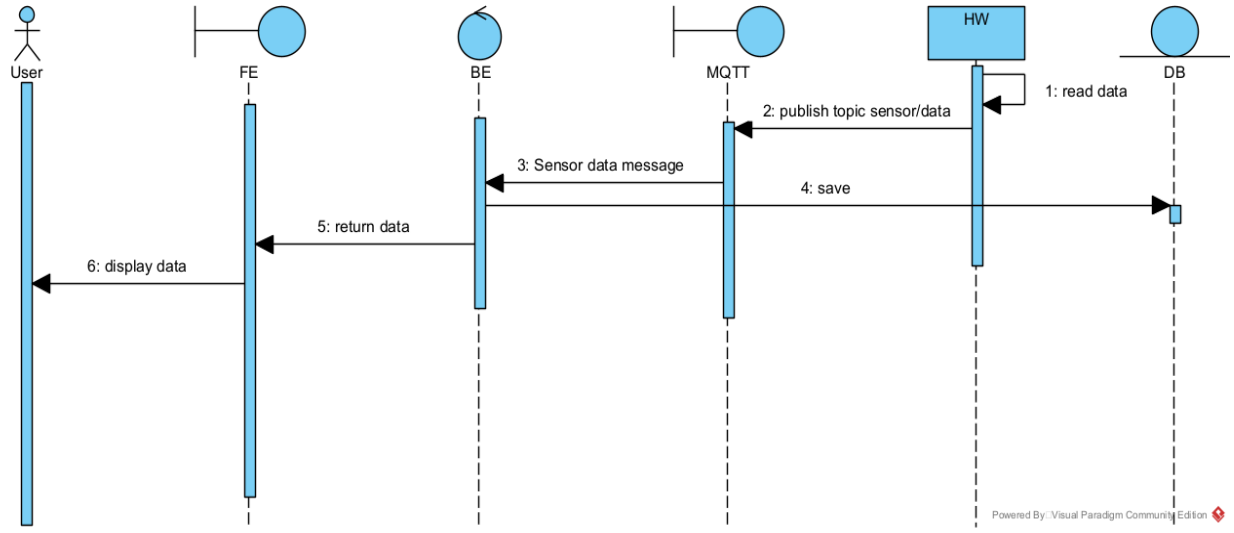
2.1.5 Luồng dữ liệu trong hệ thống

Quy trình hoạt động tổng thể của hệ thống:

1. Cảm biến thu thập dữ liệu → gửi về vi điều khiển
2. Vi điều khiển publish dữ liệu lên MQTT Broker
3. Backend subscribe và nhận dữ liệu
4. Backend xử lý và lưu vào database
5. Backend gửi dữ liệu realtime lên Frontend qua WebSocket
6. Người dùng theo dõi và gửi lệnh điều khiển từ giao diện
7. Backend publish lệnh xuống MQTT → thiết bị nhận và thực thi

2.2 Usecase

2.2.1 Luồng theo dõi cảm biến thời gian thực

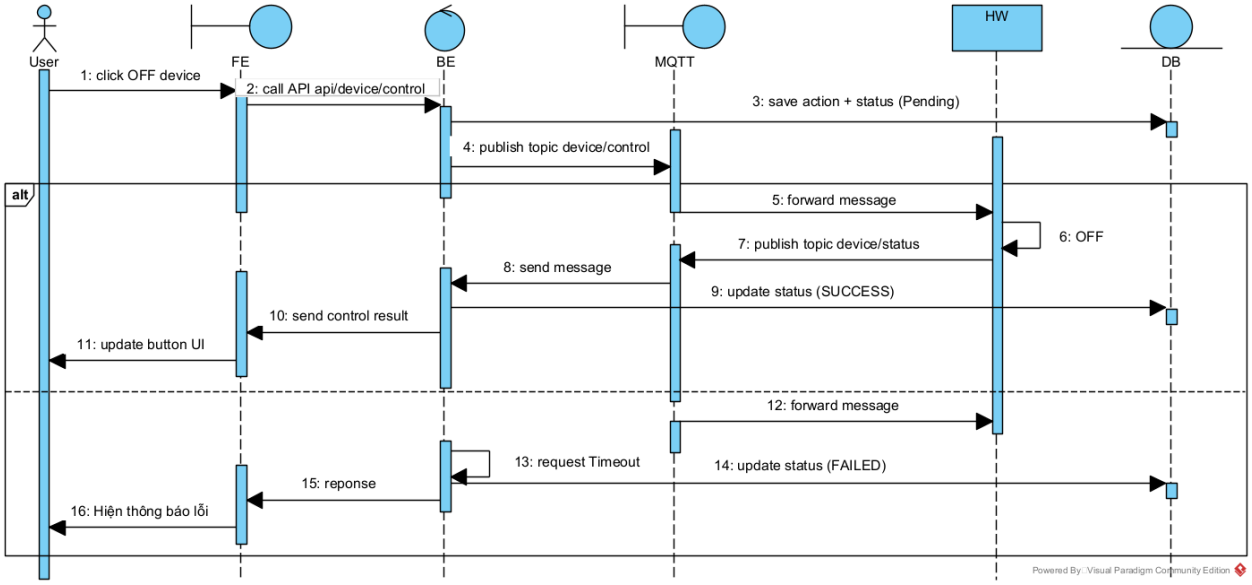


Hình 2.2 Luồng theo dõi cảm biến

Biểu đồ này mô tả luồng dữ liệu tự động từ phần cứng đẩy lên hệ thống theo thời gian thực.

- Bước 1: Sensor đọc từ liệu từ chính nó.
- Bước 2: Sensor thực hiện publish dữ liệu vào topic sensor/data thông qua MQTT Broker.
- Bước 3: BE (đã subscribe topic) nhận được tin nhắn chứa dữ liệu cảm biến.
- Bước 4: BE thực hiện lưu trữ (Save) các dữ liệu này vào Database (DB).
- Bước 5: Đồng thời, BE đẩy dữ liệu về cho FE.
- Bước 6: FE cập nhật và hiển thị dữ liệu mới nhất lên màn hình để người dùng theo dõi trực tiếp.

2.2.2 Luồng điều khiển thiết bị



Hình 2.3 Luồng điều khiển thiết bị

+ Luồng chính (Thành công):

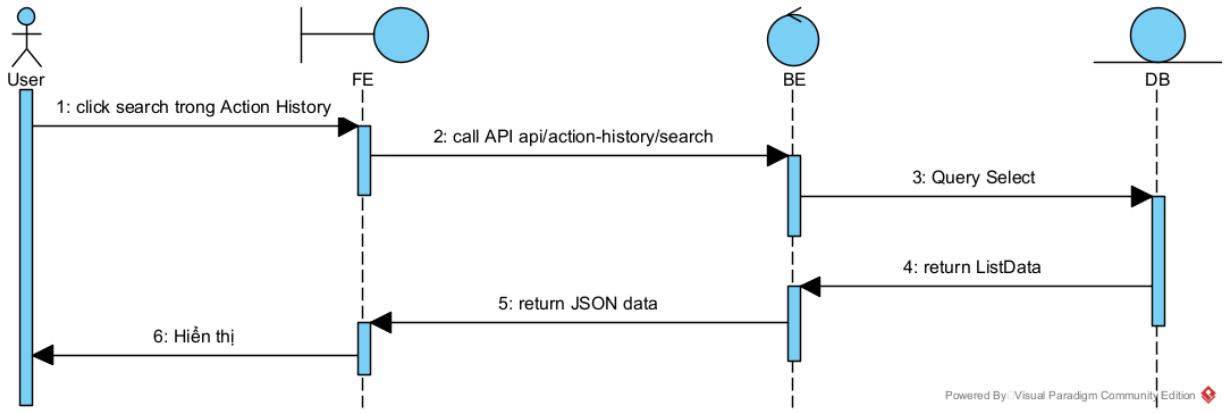
- Bước 1 - 2: User nhấn nút "OFF" thiết bị, FE gọi API điều khiển đến BE.
- Bước 3: BE lưu trạng thái yêu cầu vào DB với trạng thái ban đầu là Pending.
- Bước 4 - 5: BE publish (gửi) một tin nhắn vào topic device/control thông qua MQTT Broker. Broker sau đó forward (chuyển tiếp) thông điệp này đến thiết bị phần cứng (HW).
- Bước 6: Thiết bị thực hiện lệnh.
- Bước 7 - 8: HW phản hồi lại trạng thái mới thông qua MQTT, gửi tin nhắn ngược lại cho BE qua topic device/status.
- Bước 9: BE cập nhật trạng thái mới của thiết bị vào DB.
- Bước 10 - 11: BE gửi kết quả điều khiển về cho FE để cập nhật lại trạng thái nút bấm trên giao diện người dùng.

+ Luồng thay thế (Xử lý lỗi):

- Bước 12: MQTT chuyển tiếp thông điệp đến thiết bị.
- Bước 13-14: Nếu có vấn đề, hệ thống ghi nhận trạng thái thất bại vào DB.

- Bước 15-16: BE gửi phản hồi cho FE, và FE cập nhật trạng thái nút hiển thị thông báo cho người dùng.

2.2.3 Luồng tra cứu lịch sử thao tác



Hình 24 Luồng tra cứu lịch sử thao tác

+ Bước 1: User nhấn nút "Search" trong giao diện Action History trên Frontend (FE).

+ Bước 2: FE gửi yêu cầu gọi API api/action-history/search đến Backend (BE).

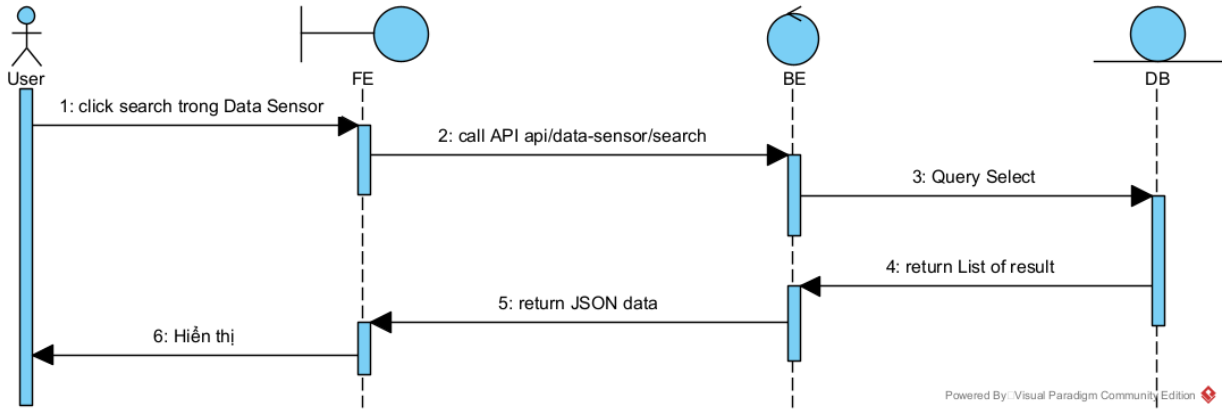
+ Bước 3: BE thực hiện lệnh Query Select xuống Database (DB) để lấy dữ liệu lịch sử.

+ Bước 4: DB trả kết quả danh sách dữ liệu (return ListData) cho BE.

+ Bước 5: BE xử lý và trả về dữ liệu dưới dạng JSON cho FE.

+ Bước 6: FE nhận dữ liệu và hiển thị (Render) lên màn hình cho User.

2.2.4 Luồng tra cứu dữ liệu cảm biến



Hình 2.5 Luồng tra cứu dữ liệu cảm biến

+ Bước 1: User nhấn nút "Search" trong màn hình Data Sensor trên FE.

+ Bước 2: FE gọi API api/data-sensor/search gửi đến BE.

+ Bước 3: BE truy vấn cơ sở dữ liệu (DB) bằng lệnh Query Select.

+ Bước 4: DB trả về danh sách kết quả đo đạc của cảm biến cho BE.

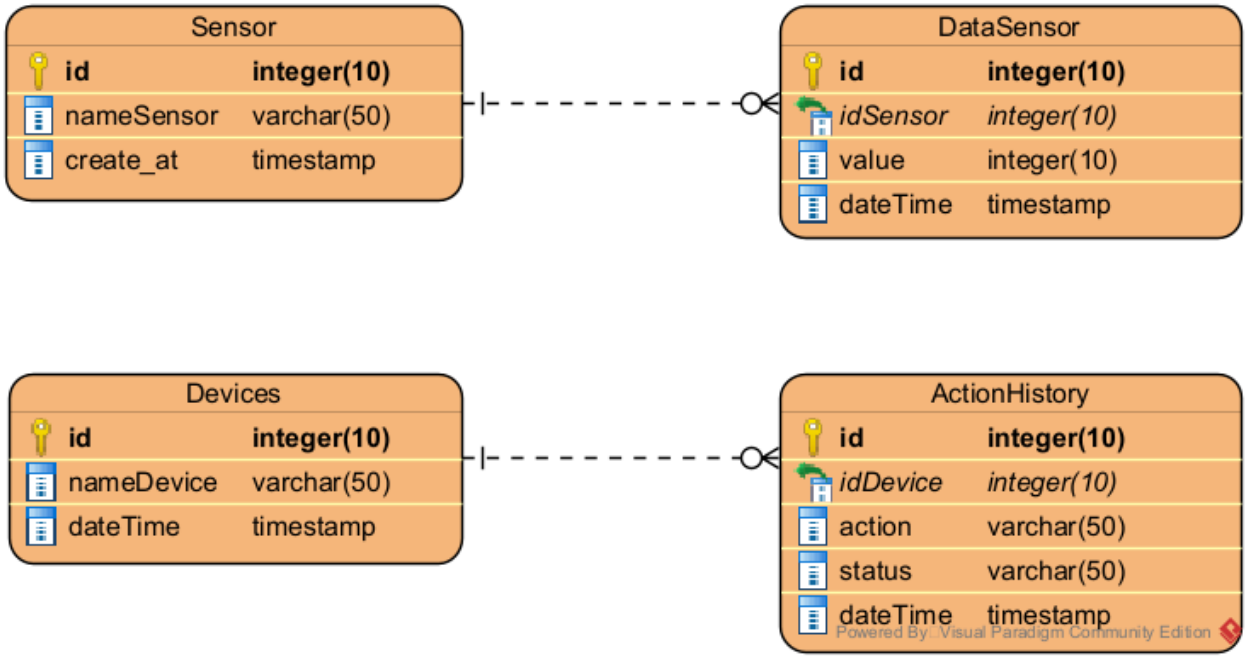
+ Bước 5: BE đóng gói và trả dữ liệu JSON về cho FE.

+ Bước 6: FE thực hiện hiển thị các thông số lên giao diện cho User.

CHƯƠNG III: XÂY DỰNG HỆ THỐNG

3.1 Thiết kế cơ sở dữ liệu

- Cơ sở dữ liệu gồm 4 bảng chính:



Hình 3.1 Thiết kế cơ sở dữ liệu

+ Bảng Sensor: Lưu trữ thông tin danh mục các loại cảm biến.
    - id (PK): Mã định danh duy nhất của cảm biến.
    - nameSensor: Tên của cảm biến.
    - create_at: Thời gian khởi tạo cảm biến trong hệ thống.

+ Bảng DataSensor: Lưu trữ các giá trị đo đạc thực tế từ cảm biến.
    - id (PK): Mã định danh bản ghi dữ liệu.
    - idSensor (FK): Khóa ngoại liên kết tới bảng Sensor.
    - value: Giá trị đo được từ cảm biến.
    - dateTime: Thời điểm ghi nhận dữ liệu.

+ Bảng Devices: Lưu trữ thông tin về các thiết bị có thể điều khiển.
    - id (PK): Mã định danh duy nhất của thiết bị.
    - nameDevice: Tên thiết bị.
    - dateTime: Thời gian thiết bị được thêm vào hệ thống.

+ Bảng ActionHistory: Lưu trữ nhật ký các thao tác điều khiển thiết bị.

- id (PK): Mã định danh bản ghi lịch sử.
- idDevice (FK): Khóa ngoại liên kết tới bảng Devices.
- action: Hành động thực hiện.
- status: Trạng thái của hành động.
- dateTime: Thời điểm thực hiện thao tác.

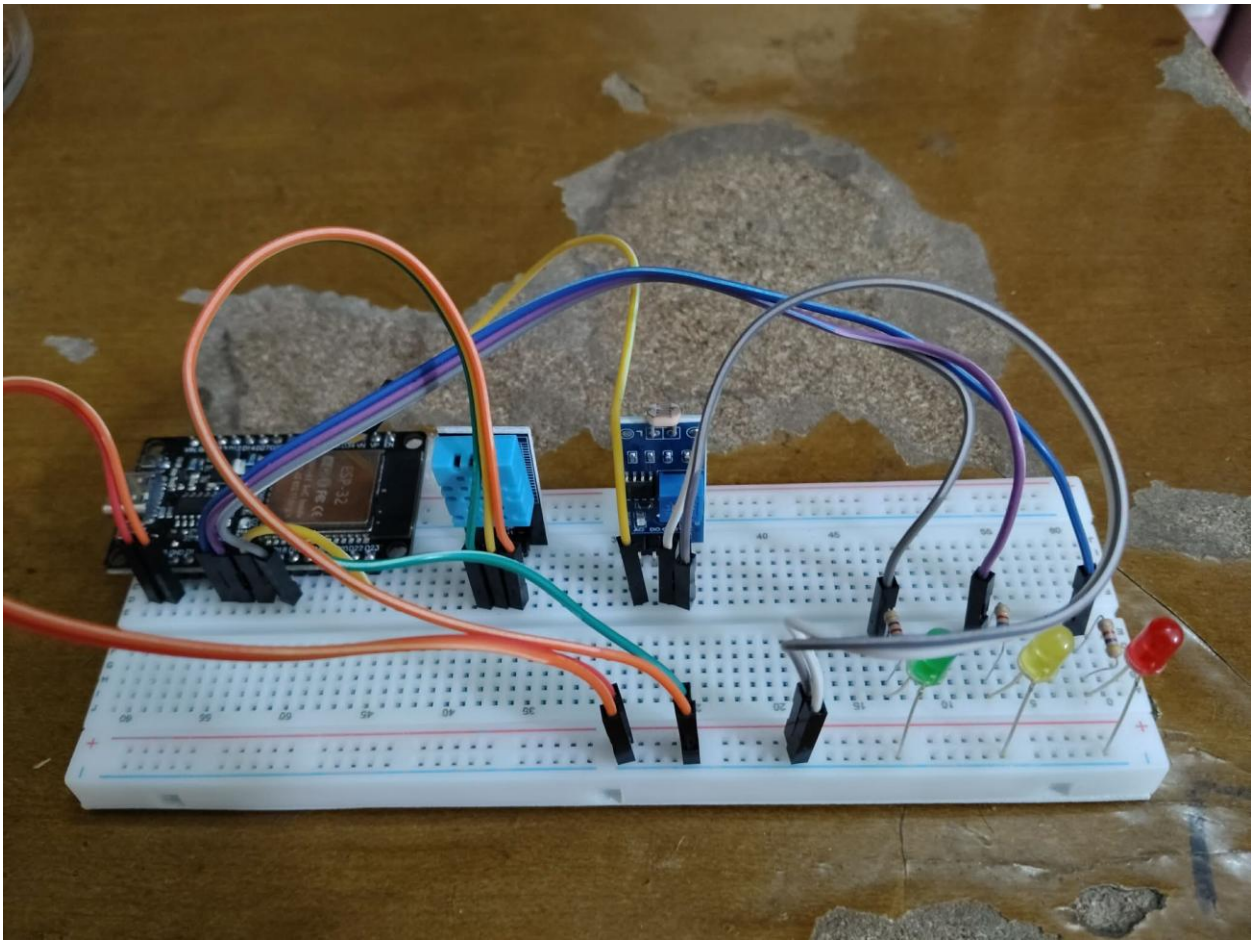
3.2 Thiết kế API

Các API đã triển khai

API	Method	Path	Tham số vào	Mô tả
Điều khiển thiết bị	POST	/api/device/control	led, action	Tìm thiết bị theo tên, tạo lịch sử với trạng thái Pending, lên lịch đổi sang Failed sau 5 giây nếu chưa hoàn tất, rồi gửi MQTT tới topic device/control.
Tìm dữ liệu cảm biến	GET	/api/data-sensor/search	nameSensor, value, dateTime, sensorId, page, size, sortBy, direction	Lọc dữ liệu sensor theo điều kiện, có phân trang và sắp xếp. dateTime hỗ trợ định dạng yyyy-MM-ddTHH:mm:ss hoặc yyyy-MM-dd HH:mm:ss; sai định dạng trả 400.
Tìm lịch sử thao tác	GET	/api/action-history/search	nameDevice, status, action, date, dateTime, page, size, sortBy, direction	Lọc lịch sử theo thiết bị, trạng thái, hành động, thời gian. dateTime ưu tiên hơn date; sai định dạng trả 400.

3.3 Triển khai phần cứng

3.3.1 Triển khai thực tế



Hình 3.2 Triển khai phần cứng thực tế

3.3.2 Lập trình phần cứng

```
#include <WiFi.h>
#include <PubSubClient.h>
#include "DHT.h"
#include <ArduinoJson.h>

// CẤU HÌNH KẾT NỐI
const char* ssid = "vivo";
const char* password = "6868686868";
const char* mqtt_server = "10.96.94.220"; //"172.20.10.8"  192.168.0.104 ;
const int mqtt_port = 1884;

// ĐỊNH NGHĨA CHÂN
#define LED_DO 16
#define LED_XANH 19
#define LED_VANG 17
#define DHTPIN 5
#define DHTTYPE DHT11
#define LIGHT_PIN 35

DHT dht(DHTPIN, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);
```

Hình 3.3 Cấu hình hết nối phần cứng

- Cấu hình thư viện và khởi tạo hệ thống: Bao gồm việc khai báo các thư viện cần thiết như PubSubClient và ArduinoJson, đồng thời thiết lập các thông số ban đầu cho hệ thống như chân GPIO, Wi-Fi, MQTT Broker.

```
void callback(char* topic, byte* payload, unsigned int length) {
  String message = "";
  for (int i = 0; i < length; i++) message += (char)payload[i];

  String strTopic = String(topic);
  if (strTopic == "device/control") {
    message.toLowerCase();
    message.trim();

    int spaceIndex = message.indexOf(' ');
    if (spaceIndex == -1) return;

    String ledName = message.substring(0, spaceIndex);
    String action  = message.substring(spaceIndex + 1);

    int pin = -1;
    if (ledName == "light_bulb") pin = LED_DO;
    else if (ledName == "air_condition") pin = LED_XANH;
    else if (ledName == "fan") pin = LED_VANG;

    if (pin != -1) {
      bool state = (action == "on");
      digitalWrite(pin, state ? HIGH : LOW);

      // Gửi phản hồi trạng thái về BE
      String statusMsg = ledName + " " + action + " success";
      client.publish("device/status", statusMsg.c_str());
    }
  }
}
```

Hình 3.4 Tiếp nhận và xử lý các lệnh điều khiển

- Tiếp nhận và xử lý các lệnh điều khiển từ hệ thống Backend thông qua giao thức MQTT. Khi ESP32 nhận được message từ Broker, hàm này sẽ:
  - Phân tích nội dung lệnh
  - Xác định thiết bị cần điều khiển
  - Thực hiện bật/tắt thiết bị tương ứng
  - Gửi phản hồi trạng thái về hệ thống

```
void loop() {
  if (!client.connected()) reconnect();
  client.loop();

  static unsigned long lastMsg = 0;
  if (millis() - lastMsg > 2000) {
    lastMsg = millis();

    int lightVal = 4095 - analogRead(LIGHT_PIN);
    float h = dht.readHumidity();
    float t = dht.readTemperature();

    if (!isnan(h) && !isnan(t)) {
      // TẠO JSON DỮ LIỆU
      StaticJsonDocument<200> doc;
      doc["temperature"] = t;
      doc["humidity"] = h;
      doc["light"] = lightVal;
      doc["device_id"] = "ESP32_01";

      char buffer[200];
      serializeJson(doc, buffer);

      // Gửi lên MQTT
      client.publish("sensor/data", buffer);
      Serial.print("Sent: ");
      Serial.println(buffer);
    }
  }
}
```

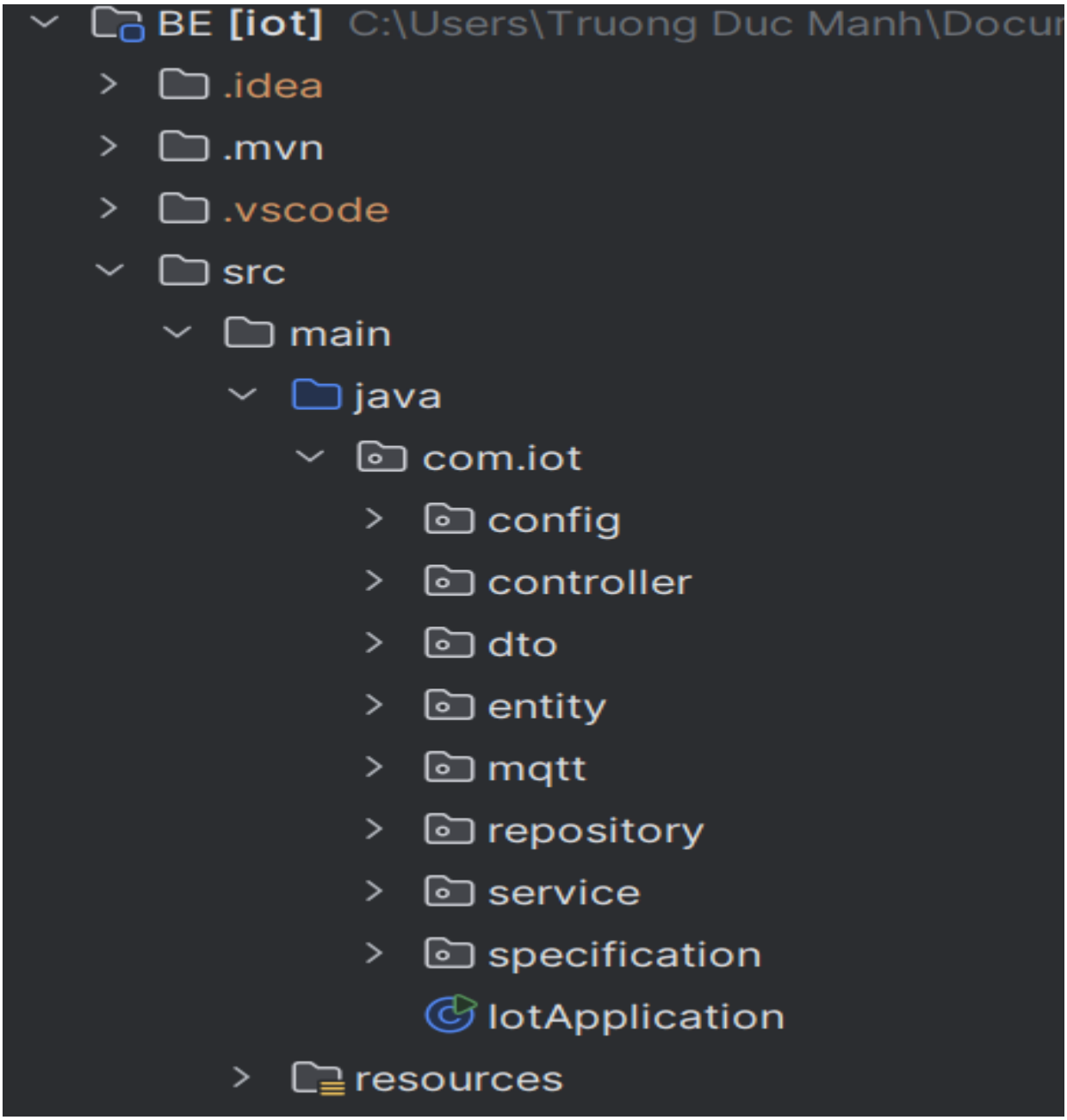
Hình 3.5 Thu thập và truyền dữ liệu cảm biến

- Thu thập và truyền dữ liệu cảm biến:
  - Duy trì kết nối MQTT với Broker
  - Lắng nghe và xử lý message từ hệ thống
  - Thu thập dữ liệu từ các cảm biến (nhiệt độ, độ ẩm, ánh sáng)
  - Đóng gói dữ liệu dưới dạng JSON
  - Gửi dữ liệu lên server thông qua MQTT

3.4 Xây dựng Backend

3.4.1 Cấu trúc lớp

Backend được chia thành 8 lớp:



Hình 3.6 Cấu trúc backend

- config: Chứa các lớp cấu hình hệ thống bao gồm cấu hình kết nối WebSocket (STOMP/SockJS) để cập nhật dữ liệu realtime và cấu hình CORS cho phép Frontend truy cập tài nguyên.

- controller: Tầng tiếp nhận các yêu cầu HTTP từ Client. Các Endpoint như /api/device/control hay /api/data-sensor/search được định nghĩa tại đây để điều hướng yêu cầu đến tầng nghiệp vụ.
- dto (Data Transfer Object): Định nghĩa các đối tượng trung gian dùng để trao đổi dữ liệu giữa các lớp, giúp tối ưu hóa cấu trúc JSON trả về cho Frontend (ví dụ: actionHistoryResponse, dataSensorResponse).
- entity: Chứa các lớp ánh xạ trực tiếp với các bảng trong cơ sở dữ liệu MySQL thông qua Spring Data JPA như Sensor, DataSensor, Devices, và ActionHistory.
- mqtt: Package chuyên trách xử lý giao thức truyền thông IoT. Bao gồm cấu hình MqttGateway để gửi lệnh và các logic xử lý tin nhắn nhận được từ Broker (ví dụ: cập nhật trạng thái thiết bị từ topic device/status).
- repository: Tầng giao tiếp dữ liệu, sử dụng các Interface kế thừa từ JpaRepository để thực hiện các thao tác truy vấn xuống cơ sở dữ liệu MySQL.
- service: Nơi thực hiện các logic nghiệp vụ phức tạp. Lớp này điều phối dữ liệu từ Repository, thực hiện tính toán, kiểm tra quyền và điều khiển thiết bị.
- specification: Triển khai các câu truy vấn động linh hoạt. Thành phần này cho phép hệ thống thực hiện tìm kiếm nâng cao với nhiều điều kiện kết hợp như lọc theo tên cảm biến, giá trị và khoảng thời gian đồng thời.
- IotApplication: Lớp khởi chạy chính (Main Class) tích hợp cơ chế tự động cấu hình của Spring Boot.

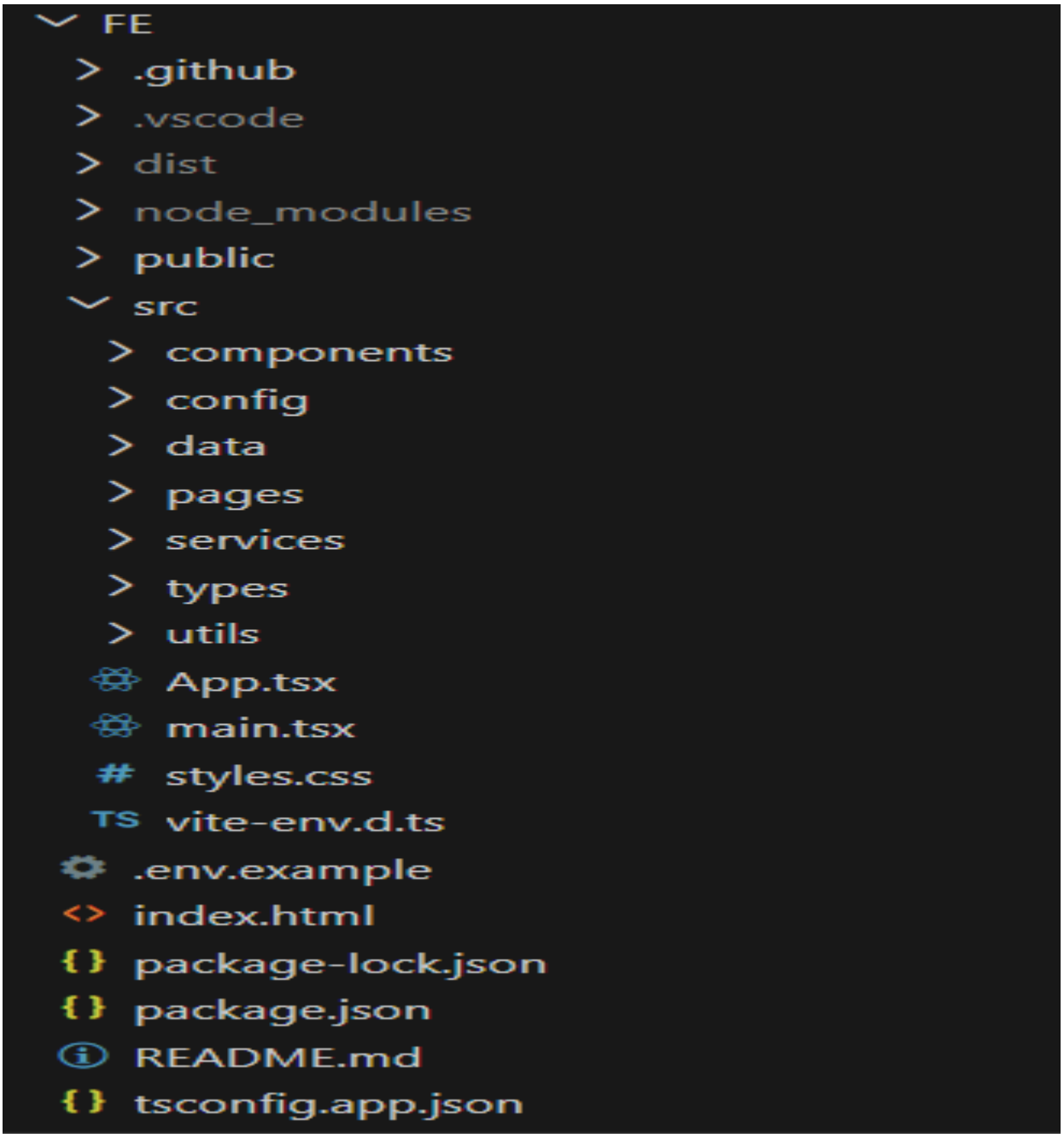
3.4.2. Các cơ chế xử lý trọng tâm

1. Cơ chế Real-time với WebSocket & MQTT:
    - Backend đóng vai trò là một MQTT Client, liên tục lắng nghe dữ liệu từ các cảm biến đẩy lên topic sensor/data.
    - Ngay khi nhận được dữ liệu, tầng service sẽ xử lý lưu trữ vào DB và đồng thời đẩy thông tin qua WebSocket để Frontend cập nhật biểu đồ ngay lập tức mà không cần F5 trang.
2. Logic điều khiển thiết bị và Timeout:
    - Khi nhận lệnh điều khiển, hệ thống tạo bản ghi nhật ký với trạng thái Pending.
    - Sử dụng ActionHistoryTimeoutService để quản lý thời gian phản hồi từ phần cứng. Nếu sau 5 giây không nhận được tín hiệu phản hồi qua MQTT, hệ thống sẽ tự động chuyển trạng thái bản ghi thành FAILED để đảm bảo tính minh bạch cho người dùng.

3.5 Xây dựng Frontend

3.5.1 Cấu trúc tổ chức mã nguồn

Ứng dụng Frontend được xây dựng dựa trên nền tảng React kết hợp với TypeScript, sử dụng công cụ xây dựng Vite để đảm bảo hiệu suất phát triển và tốc độ thực thi cao. Toàn bộ mã nguồn được tổ chức theo kiến trúc module hóa rõ rệt.



Hình 3.7 Cấu trúc frontend

Các thành phần được phân chia như sau:

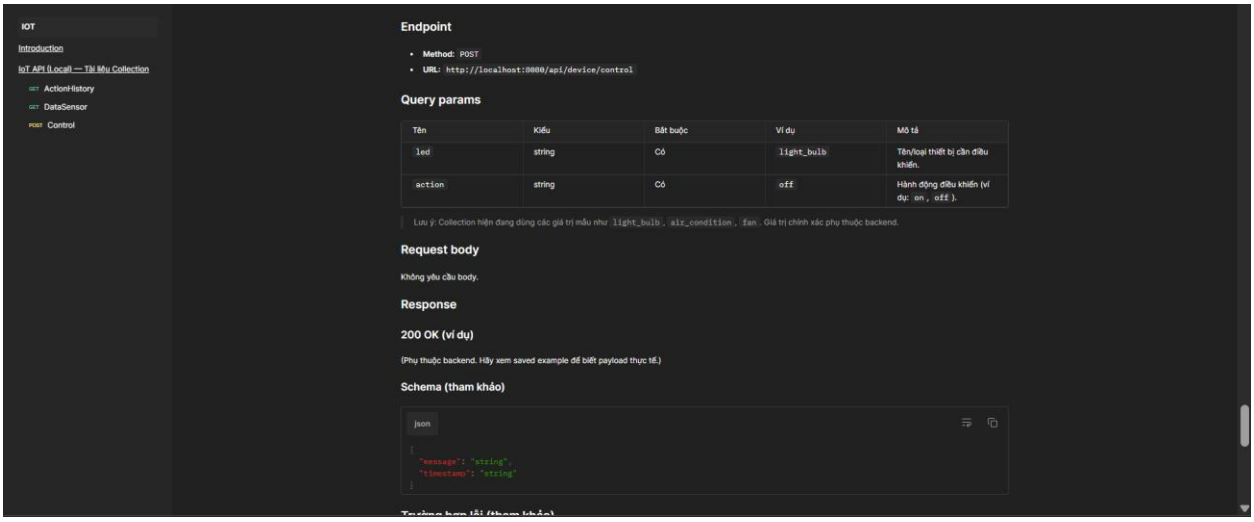
- pages: Chứa các thành phần đại diện cho các trang chính của ứng dụng, bao gồm:
    - DashboardPage.tsx: Trang điều khiển tập trung, hiển thị dữ liệu thời gian thực.
    - DataSensorPage.tsx: Trang quản lý và tra cứu lịch sử cảm biến.
    - ActionHistoryPage.tsx: Trang hiển thị nhật ký điều khiển thiết bị.
    - ProfilePage.tsx: Trang thông tin cá nhân và liên kết dự án.
- components: Chứa các thành phần UI có khả năng tái sử dụng (Reusable Components), giúp giao diện đồng nhất:
    - ChartCard.tsx: Thành phần hiển thị biểu đồ dữ liệu.
    - DataTable.tsx: Bảng hiển thị dữ liệu có tính năng phân trang và sắp xếp.
    - DeviceControlCard.tsx: Thẻ điều khiển bật/tắt thiết bị tích hợp trạng thái phản hồi.
    - Sidebar.tsx và PageHeader.tsx: Các thành phần điều hướng và tiêu đề hệ thống.
- services: Tầng xử lý logic kết nối dữ liệu:
    - iotApi.ts: Định nghĩa các hàm gọi REST API đến Backend.
    - iotRealtime.ts: Quản lý kết nối WebSocket/SockJS để nhận dữ liệu realtime.
    - http.ts: Cấu hình nền tảng (ví dụ: Axios instance) để giao tiếp HTTP.
- types: Định nghĩa các Interface và Type trong TypeScript (như iot.ts), đảm bảo tính chặt chẽ về dữ liệu và hạn chế lỗi trong quá trình phát triển.
- config: Chứa các tệp cấu hình môi trường và hằng số hệ thống (iot.ts).

3.5.2. Công nghệ và Kỹ thuật triển khai trọng tâm

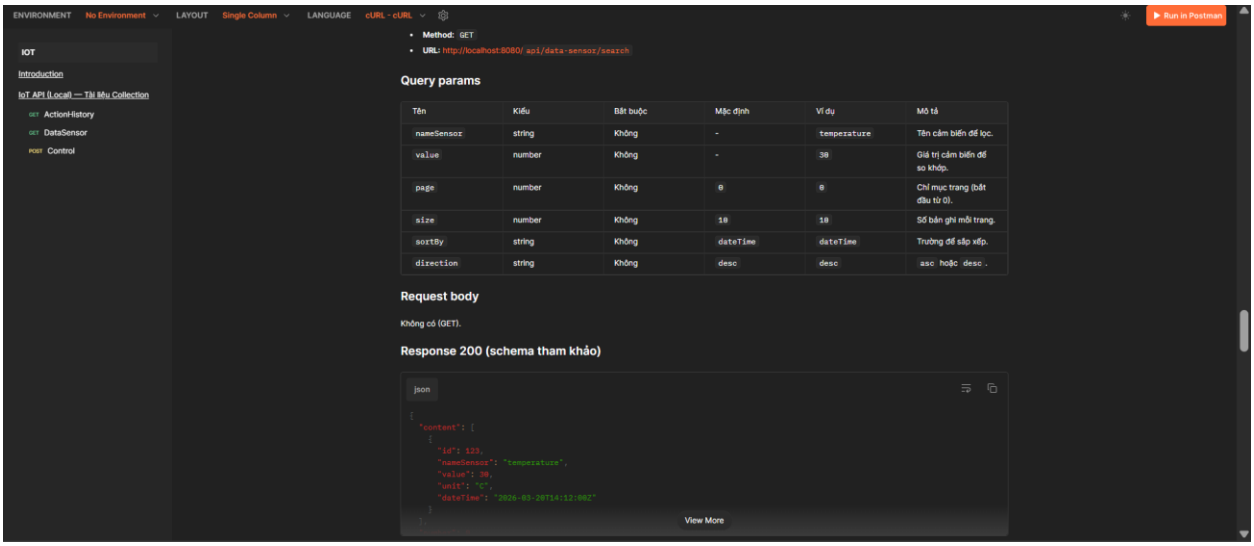
1. Quản lý trạng thái và Dữ liệu thực tế (Real-time Integration):
    - Ứng dụng sử dụng SockJS và giao thức STOMP để thiết lập kênh liên lạc hai chiều với Backend.
    - Dữ liệu cảm biến từ topic MQTT được Backend đẩy qua WebSocket sẽ được Frontend cập nhật trực tiếp vào trạng thái (state) của React, giúp biểu đồ và các thẻ số liệu thay đổi ngay lập tức mà không cần tải lại trang.
2. Tối ưu hóa trải nghiệm người dùng (UX/UI):
    - Lazy Loading: Các trang trong thư mục pages được cấu hình để chỉ tải khi cần thiết, giúp giảm kích thước tệp tin ban đầu và tăng tốc độ truy cập.
    - Responsive Design: Giao diện được thiết kế để hiển thị tốt trên nhiều kích thước màn hình khác nhau.

- Xử lý Trạng thái Đợi (Pending State): Khi thực hiện lệnh điều khiển, DeviceControlCard sẽ hiển thị trạng thái đang xử lý, phản ánh đúng quy trình xác nhận từ thiết bị phần cứng.

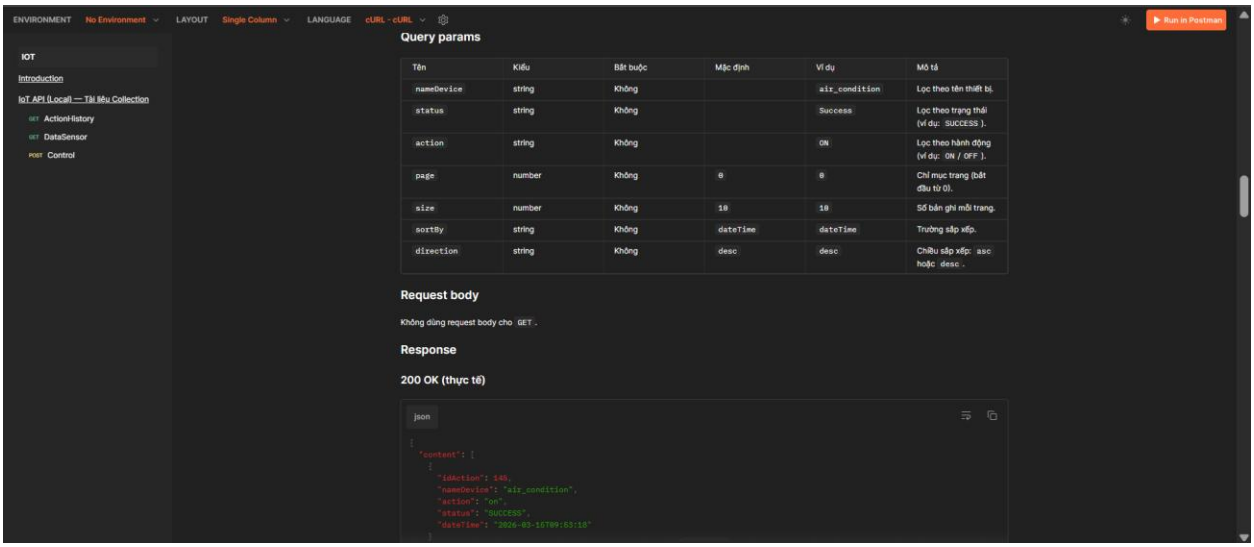
3.6 API document (Postman)



Hình 3.8 API document (Điều khiển thiết bị)



Hình 3.9 API document (Tìm kiếm dữ liệu cảm biến



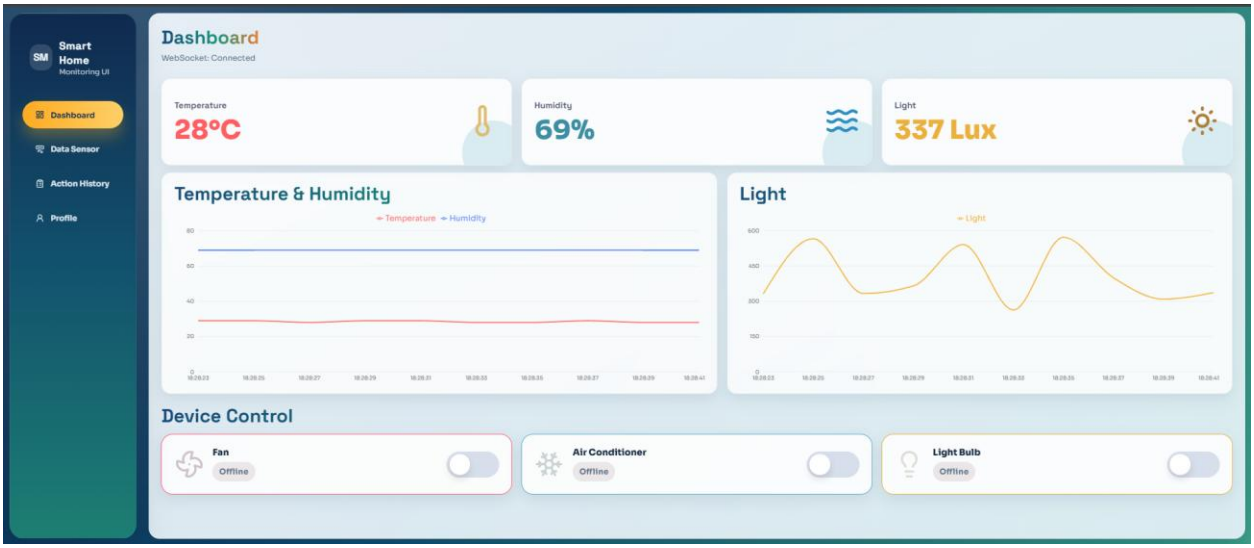
Hình 3.10 API document (Tìm kiếm lịch sử điều khiển)

CHƯƠNG IV: KẾT QUẢ THỰC HIỆN

Giao diện của hệ thống được xây dựng theo phong cách dashboard giám sát IoT hiện đại, bố cục rõ ràng và đồng nhất giữa các trang. Toàn bộ ứng dụng sử dụng một thanh điều hướng bên trái cố định để chuyển giữa các chức năng chính gồm Dashboard, Data Sensor, Action History và Profile. Khu vực nội dung chính nằm bên phải, hiển thị từng màn hình theo mục đích sử dụng.

Thiết kế giao diện tập trung vào khả năng quan sát nhanh dữ liệu thời gian thực, tra cứu lịch sử và điều khiển thiết bị. Màu sắc chủ đạo là các tông xanh, trắng và vàng cam, tạo cảm giác hiện đại, dễ nhìn và phù hợp với lĩnh vực giám sát hệ thống. Các thành phần giao diện được tổ chức dưới dạng các khối rõ ràng, bo góc lớn, có đổ bóng nhẹ, giúp tổng thể vừa chuyên nghiệp vừa trực quan.

4.1 Giao diện Dashboard



Hình 4.1 Giao diện Dashboard thực tế

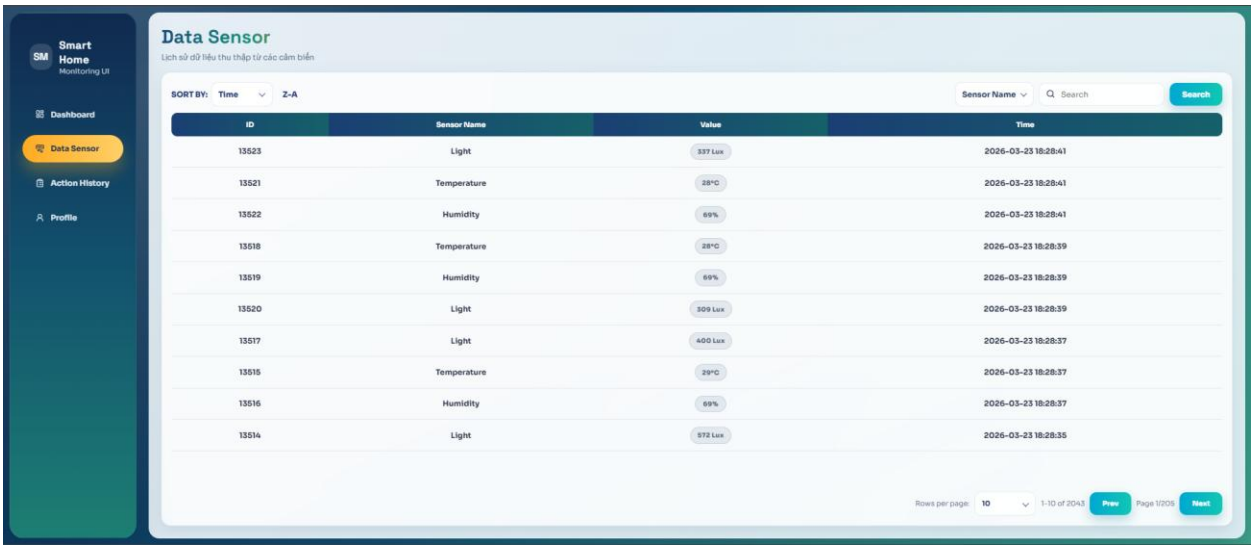
Trang Dashboard là màn hình trung tâm của hệ thống, dùng để theo dõi dữ liệu cảm biến và trạng thái thiết bị theo thời gian thực. Đây là trang quan trọng nhất vì thể hiện toàn bộ tình trạng hiện tại của hệ thống IoT.

Phần trên của trang hiển thị ba thẻ chỉ số chính gồm nhiệt độ, độ ẩm và ánh sáng. Mỗi thẻ có số liệu lớn, màu sắc khác nhau và biểu tượng minh họa riêng, giúp người dùng nhận biết nhanh giá trị hiện tại. Ở giữa trang là hai biểu đồ đường, trong đó một biểu đồ thể hiện sự thay đổi của nhiệt độ và độ ẩm, còn biểu đồ còn lại thể hiện cường độ ánh sáng theo thời gian. Nhờ đó, người dùng có thể quan sát xu hướng biến động thay vì chỉ xem giá trị tức thời.

Phần dưới của Dashboard là khu vực điều khiển thiết bị gồm quạt, điều hòa và đèn. Mỗi thiết bị được đặt trong một card riêng, có biểu tượng, trạng thái online/offline và công tắc bật/tắt. Giao diện này cho phép người dùng thao tác điều khiển trực tiếp, đồng thời hiển thị phản hồi trạng thái

để biết lệnh đã được xử lý hay chưa. Nhìn chung, Dashboard là màn hình thể hiện đầy đủ tính giám sát và điều khiển của hệ thống.

4.2 Giao diện DataSensor



Hình 4.2 Giao diện DataSensor thực tế

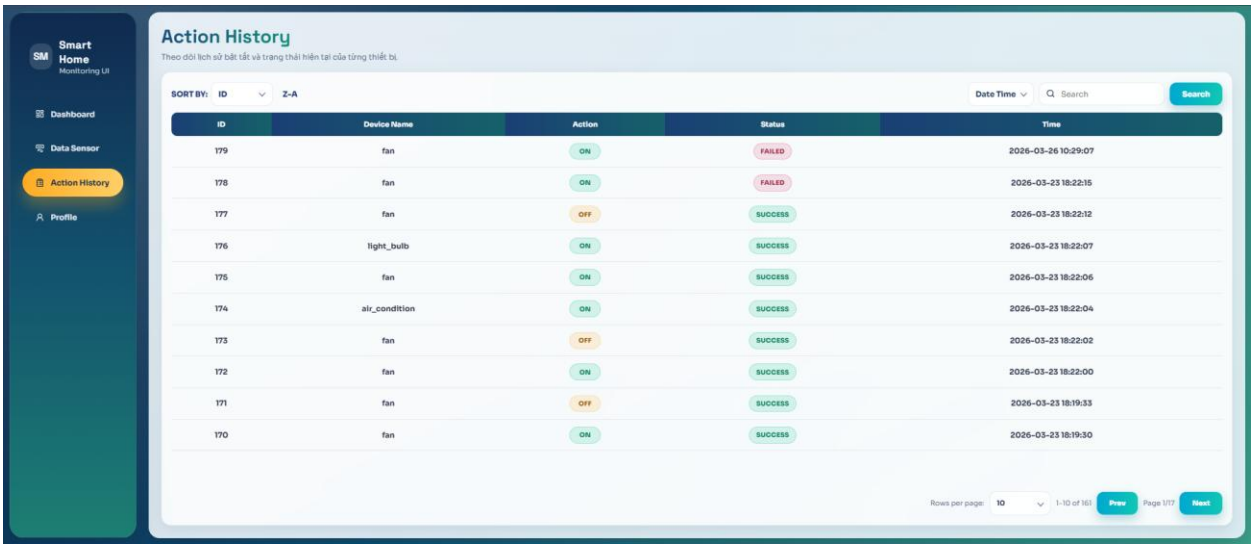
Trang Data Sensor được sử dụng để tra cứu lịch sử dữ liệu thu thập từ các cảm biến. Giao diện trang này tập trung vào bảng dữ liệu và các công cụ lọc, phù hợp với mục đích xem lại thông tin đã ghi nhận.

Phía trên bảng có khu vực tìm kiếm và sắp xếp dữ liệu. Người dùng có thể tìm theo tên cảm biến, giá trị hoặc thời gian. Ngoài ra còn có chức năng thay đổi cột sắp xếp, đảo chiều sắp xếp và phân trang. Điều này giúp việc truy xuất dữ liệu trở nên linh hoạt, đặc biệt khi số lượng bản ghi rất lớn.

Bảng dữ liệu hiển thị các cột chính gồm ID, Sensor Name, Value và Time. Giá trị cảm biến được hiển thị dưới dạng nhãn màu, tùy theo loại dữ liệu và ngưỡng giá trị. Ví dụ, nhiệt độ cao sẽ có màu cảnh báo, độ ẩm hoặc ánh sáng cũng được phân biệt bằng màu sắc phù hợp. Cách trình bày này giúp dữ liệu dễ đọc hơn và hỗ trợ người dùng nhận biết nhanh các giá trị bất thường.

Có thể đánh giá rằng Data Sensor là trang phục vụ cho việc tra cứu và phân tích dữ liệu lịch sử. Trang này không chỉ hiển thị số liệu mà còn hỗ trợ người dùng lọc, tìm kiếm và đọc dữ liệu hiệu quả hơn.

4.3 Giao diện ActionHistory



Hình 4.3 Giao diện ActionHistory thực tế

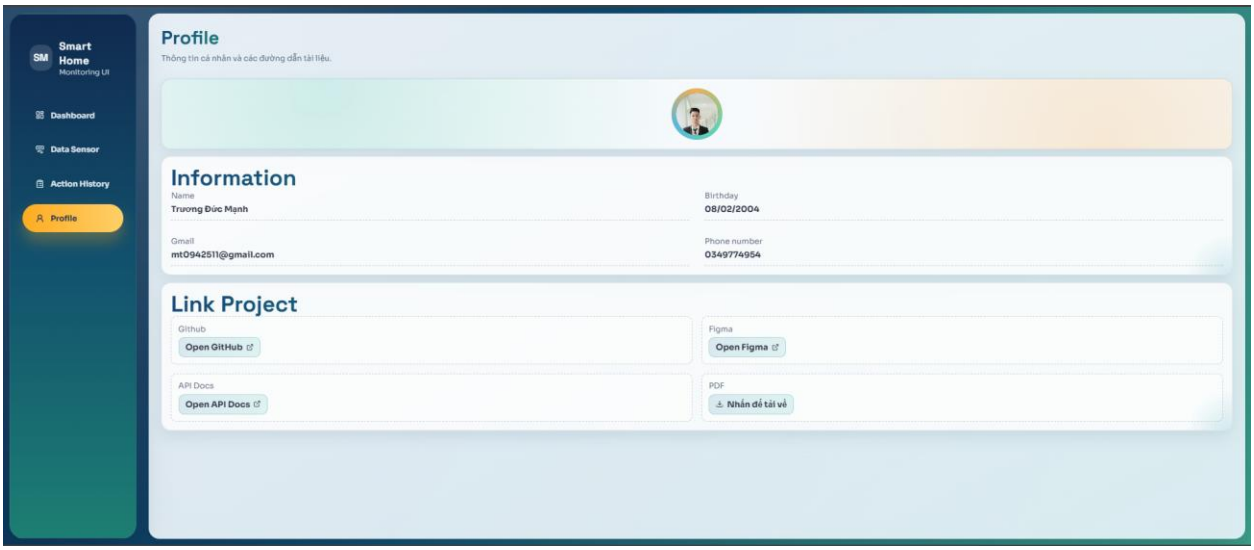
Trang Action History dùng để theo dõi lịch sử điều khiển thiết bị trong hệ thống. Đây là trang quan trọng về mặt nghiệp vụ vì nó phản ánh toàn bộ thao tác bật/tắt thiết bị và trạng thái phản hồi tương ứng.

Tương tự trang Data Sensor, phần chính của trang này là một bảng dữ liệu có hỗ trợ tìm kiếm, sắp xếp và phân trang. Người dùng có thể tìm lịch sử theo tên thiết bị, hành động, trạng thái hoặc thời gian. Bảng dữ liệu hiển thị các cột gồm ID, Device Name, Action, Status và Time.

Điểm nổi bật của trang này là cột Action và Status được hiển thị bằng nhãn màu. Hành động ON và OFF có màu khác nhau, còn trạng thái SUCCESS, FAILED hoặc PENDING cũng được phân biệt rõ. Nhờ vậy, người dùng có thể quan sát nhanh lệnh nào đã thành công, lệnh nào thất bại và thời điểm xảy ra thao tác.

Về mặt hệ thống, Action History đóng vai trò là nhật ký hoạt động, giúp kiểm tra lại các lệnh điều khiển đã gửi xuống thiết bị. Đây là trang rất hữu ích khi cần theo dõi lỗi, đối chiếu phản hồi hoặc đánh giá độ ổn định của thiết bị trong quá trình vận hành.

4.4 Giao diện Profile



Hình 4.4 Giao diện Profile thực tế

Trang Profile là trang thông tin cá nhân và tài liệu dự án. Không giống các trang còn lại tập trung vào giám sát và điều khiển, trang này phục vụ mục đích giới thiệu người thực hiện cũng như cung cấp các liên kết liên quan đến dự án.

Phần đầu của trang hiển thị ảnh đại diện ở trung tâm. Bên dưới là khối Information chứa các thông tin cá nhân cơ bản như họ tên, ngày sinh, email và số điện thoại. Tiếp theo là khối Link Project, nơi chứa các liên kết quan trọng như GitHub, Figma, API Docs và file PDF. Các nút liên kết được thiết kế rõ ràng, dễ thao tác, giúp người dùng truy cập nhanh đến tài liệu hoặc mã nguồn của dự án.

Trang Profile có vai trò bổ trợ, nhưng lại rất cần thiết trong một đồ án hoàn chỉnh vì nó thể hiện đầy đủ thông tin giới thiệu và tài nguyên tham khảo. Giao diện trang này gọn gàng, ít tương tác, và phù hợp với phong cách tổng thể của hệ thống.

CHƯƠNG IV: KẾT LUẬN

5.1 Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế và triển khai, hệ thống đã hoàn thành đầy đủ các mục tiêu đề ra ban đầu:

- Về mặt kỹ thuật (Backend & Frontend): Xây dựng thành công hệ thống Backend ổn định bằng Spring Boot, tích hợp hiệu quả giao thức MQTT để giao tiếp với phần cứng và WebSocket để cập nhật dữ liệu thời gian thực.
    - Phát triển giao diện Frontend hiện đại với React và TypeScript, đảm bảo tính trực quan, tốc độ phản hồi nhanh và khả năng tương tác mượt mà trên Dashboard.
- Về mặt tính năng:
    - Giám sát thời gian thực: Thu thập chính xác các thông số nhiệt độ, độ ẩm, ánh sáng từ cảm biến và hiển thị lên biểu đồ ngay lập tức.
    - Điều khiển thiết bị: Thực hiện bật/tắt các thiết bị (Quạt, Điều hòa, Đèn) từ xa với cơ chế phản hồi trạng thái và xử lý Timeout (5 giây) để đảm bảo tính minh bạch.
    - Quản lý dữ liệu: Lưu trữ và cho phép tra cứu linh hoạt lịch sử dữ liệu cảm biến (hơn 2000 bản ghi) và nhật ký thao tác điều khiển (Success/Failed).
- Về mặt phần cứng: Triển khai thành công mô hình thực tế với vi điều khiển ESP32, kết nối ổn định với các cảm biến DHT11, Quang trở và hệ thống LED mô phỏng thiết bị.

5.2 Hạn chế

Dù đã đạt được những kết quả khả quan, hệ thống vẫn còn một số điểm cần cải thiện:

- Bảo mật: Hiện tại hệ thống chưa triển khai các lớp bảo mật nâng cao như JWT cho API hoặc SSL/TLS cho giao thức MQTT.
- Tính năng nâng cao: Hệ thống mới chỉ dừng lại ở mức giám sát và điều khiển thủ công, chưa có các kịch bản tự động hóa (ví dụ: tự bật điều hòa khi nhiệt độ > 30°C).
- Phần cứng: Mô hình triển khai trên Breadboard nên tính thẩm mỹ và độ bền trong môi trường thực tế còn hạn chế.

5.3 Hướng phát triển tương lai

Để hoàn thiện hệ thống, em dự kiến sẽ tập trung vào các hướng phát triển sau:

1. Tự động hóa (Automation): Thiết lập các luật (Rules) để hệ thống tự động điều hành thiết bị dựa trên ngưỡng dữ liệu cảm biến.

2. Tăng cường bảo mật: Tích hợp Spring Security và mã hóa dữ liệu truyền tải để đảm bảo an toàn hệ thống.
3. Mở rộng nền tảng: Phát triển ứng dụng di động (Mobile App) để tăng tính linh hoạt cho người dùng.
4. Phân tích dữ liệu: Sử dụng các thuật toán học máy để dự báo xu hướng tiêu thụ điện năng hoặc biến đổi môi trường dựa trên kho dữ liệu đã thu thập được.